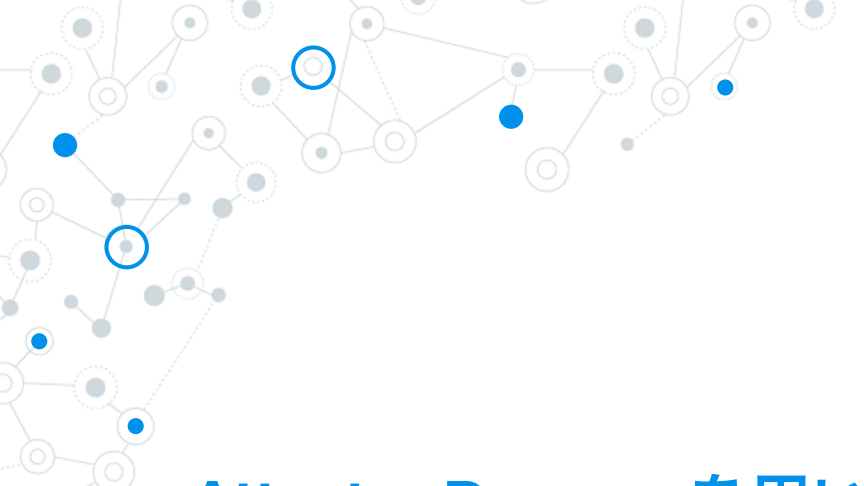




Attester Daemonを用いたアプリケーションの真正性検証の 安全性評価

佐々木怜名^{1,2} 石川裕^{2,3} 竹房あつ子^{2,4} 小口正人^{1,2}

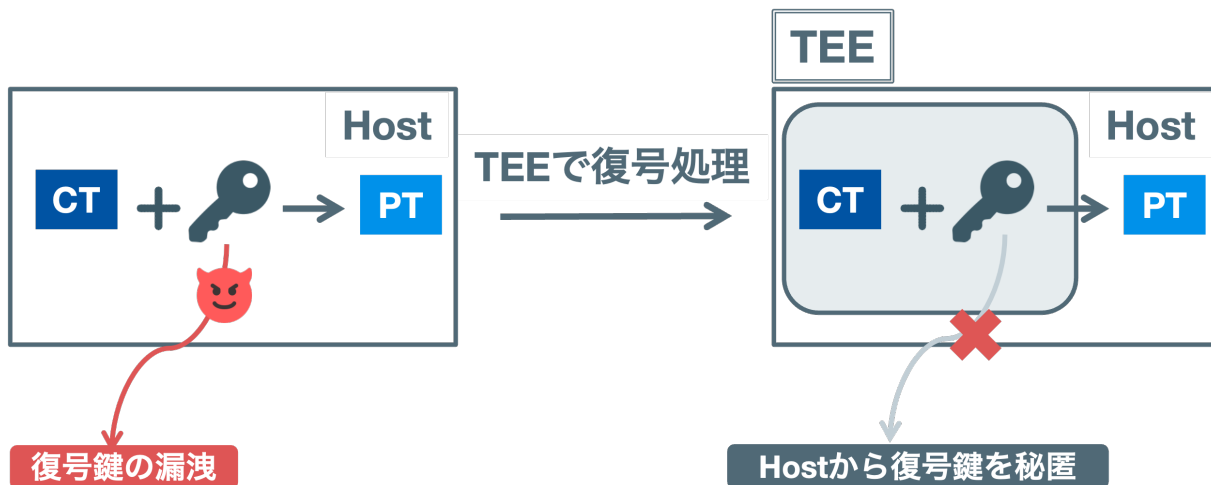
1. お茶の水女子大学 2. 情報・システム研究機構 セキュアコンピュータシステム研究開発センター
3. 大妻女子大学 4. 国立情報学研究所

本研究は、JST 経済安全保障重要技術育成プログラム【JPMJKP24U4】の支援を受けたものです。



■ TEEの暗号処理アプリケーションへの適用

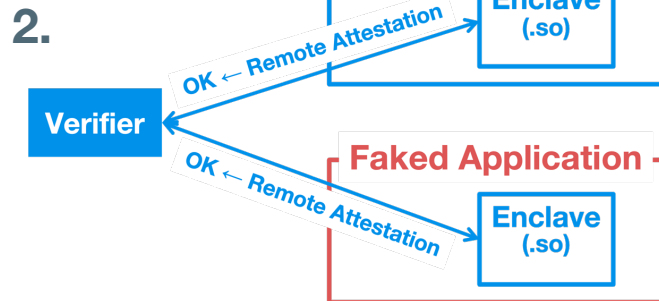
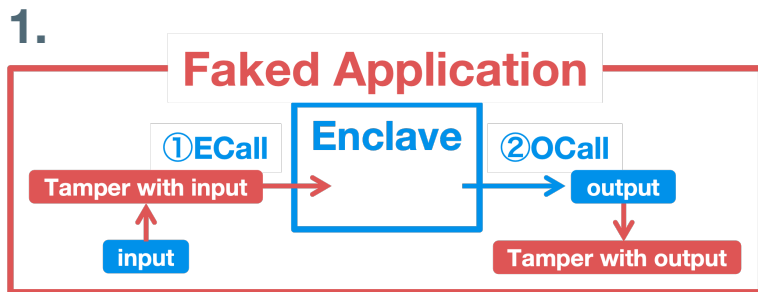
- 目的：
 - ホストOS侵害による復号鍵の漏洩を防ぎ、機密性保護の維持
 - 復号鍵の失効処理の効率化 (e.g. 属性ベース暗号) [1]
- 手法：TEE内で復号鍵を使う暗号処理を実行し、ホストから鍵を秘匿



TEEを用いたアプリケーションの課題

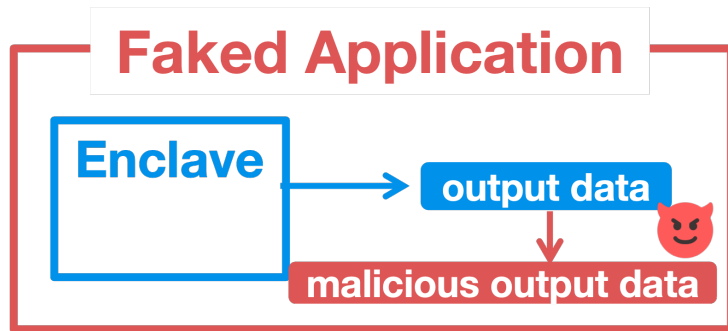
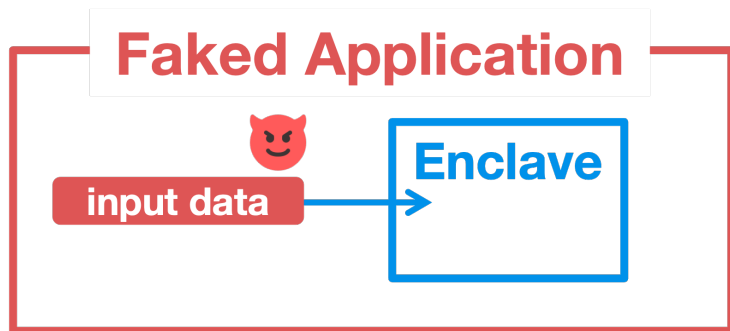
(Intel SGXが実現するTEEを対象とし、以降はSGXにおける隔離実行環境をEnclaveと呼ぶ)

1. EnclaveとHostの境界をまたぐデータは改ざん攻撃の対象となりうる^[2]
 - ECALLの引数やOCALLの戻り値が改ざんされる攻撃の対象となりやすい
2. Enclaveを呼び出すアプリケーション (Host) の真正性・完全性保証が困難
 - Enclaveのプログラム (.so形式) は可搬性があるが、任意のアプリケーションから呼び出され得る
 - 署名検証の対象となるアプリケーションが一意に定まらず、さらにそれらが依存する共有ライブラリを含めて、事前に署名・検証するのは運用上困難

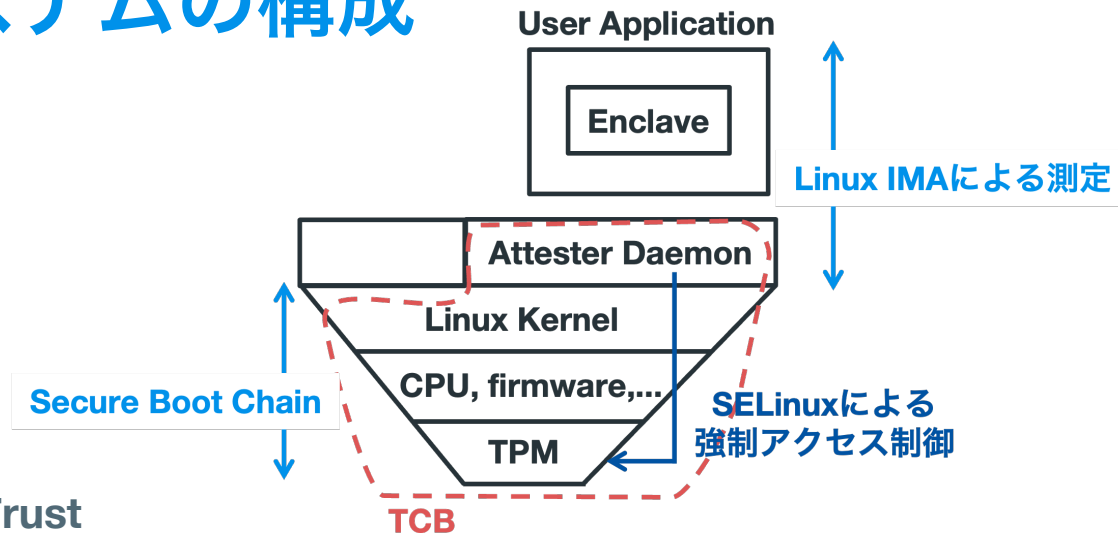


アプリケーションの真正性を検証する必要性

- アプリが不正 or 改ざんされている場合，以下の可能性がある
 - 不正な引数によるEnclave内での暗号処理の実行
 - Enclave内での暗号処理結果が不正に利用
- ✓ Enclaveを呼び出すアプリケーションの真正性と完全性を検証する仕組みが，システム全体のセキュリティ担保に不可欠



提案システムの構成



Root of Trust

- TPM: 起動時・実行時の測定値蓄積, PCR保護, Quote
- Intel SGX HW: Enclaveの実行・Local attestation

TCB (Trusted Computing Base) の構成

ファームウェア, ブートローダ, Linux Kernel, Attester Daemon

- Secure Boot Chainにより, ブートローダからLinux Kernelまでを測定
- Attester Daemon: 起動時にLinux IMAの測定
- SELinux^[3]により, TPMへのアクセス権限をAttester Daemonのみに限定

提案システムの概要

■ Intel SGXを用いる暗号アプリケーション

アプリケーションのアテステーション後に, Verifierを兼ねる鍵管理サーバ (KMS) がEnclaveに鍵を配布し, Enclaveで復号処理を実行

○ アテステーション

ある特定のプラットフォームについて信頼されたHW上で改ざんされずに正しく実行していることを証明

■ システムデプロイ

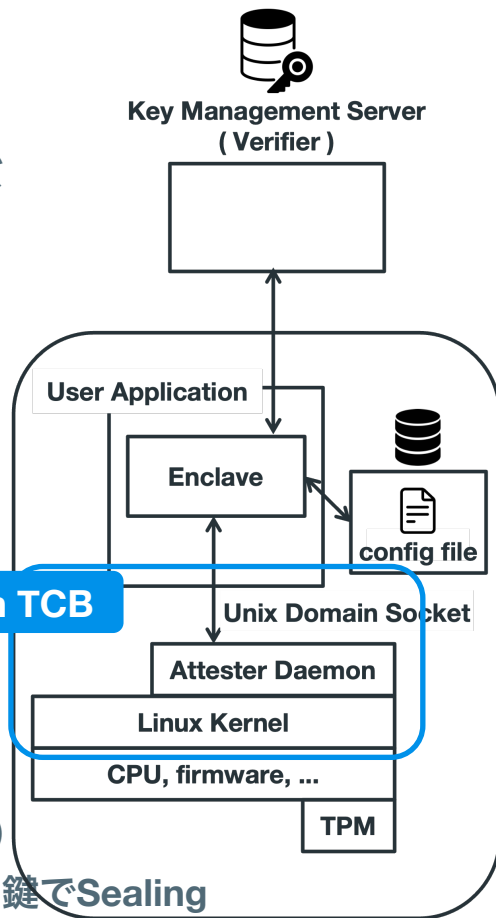
Secure bootとLinux IMAによるBoot chainからアプリケーションまでの測定 (measurement) と, TPMのPCRに測定値をextend

■ アプリケーションのアテステーション

Attester Daemonがエビデンスを生成

- KMS (Verifier)とEnclave間でRA-TLS通信のハンドシェイク時に, Enclaveのアテステーションとともに実施 (Remote Attestation: RA)
config fileにエビデンスや復号鍵などを保存, fileをEnclaveの固有の鍵でSealing
- ネットワーク遮断時は, App実行時にEnclaveがVerifierとなりLocal Attestation

Attestation TCB



User ApplicationのAttestationのEvidence

■ RA-TLSの拡張

- SGX quote
- TPM2 Quote

- quote

Attestation TCBのPCR, nonceなどを含む

TPM2_ATTEST構造体

- quote with TPM sign

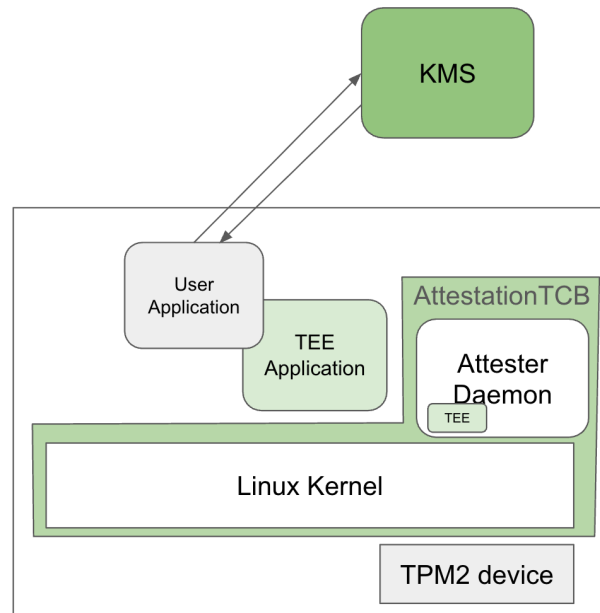
TPM2_ATTEST構造体をAKで署名

- user application hash (=app-hash)

Daemonが測定したAppのハッシュ

- Hash(app-hash | sealingkey_{Attester Daemon} | nonce)

DaemonのTEEでEGETKEYをして得たCPU固有の秘密値由来のID



提案システムの攻撃モデル [1/3]

■ Replay Attack

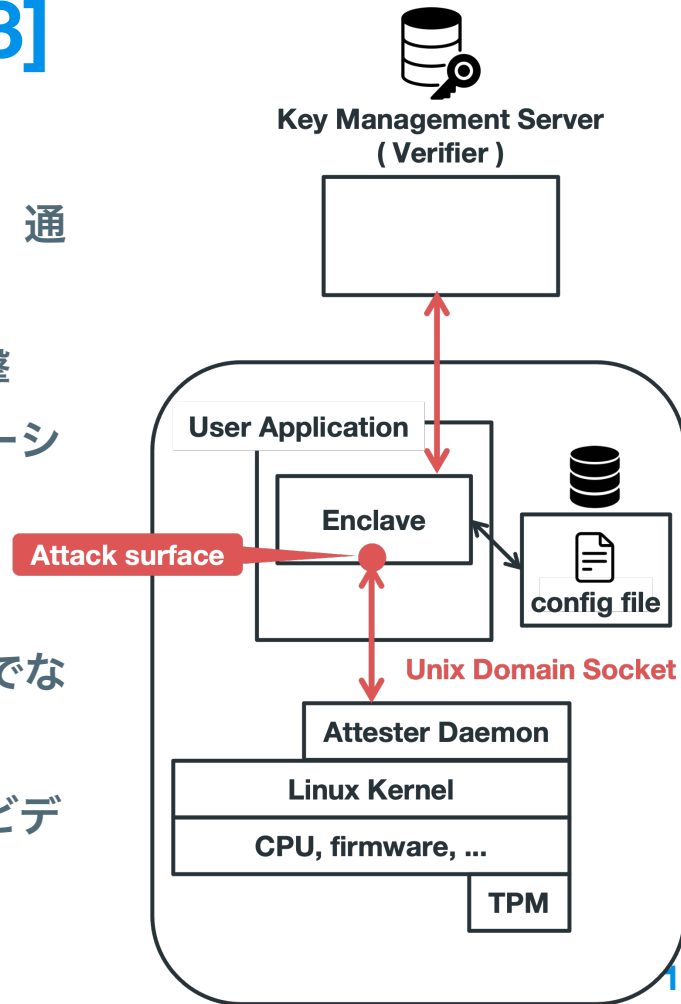
通信路でメッセージを盗聴・保存し、後に再送して、通信相手に正当なものと誤認させる攻撃

- Enclave↔Attester Daemon間の通信路への攻撃
- 過去のエビデンスを正当なものとしてアテストーション時に使用されるリスク

■ 改ざん・偽造

エビデンス中のデータを別のものに書き換え・正当でないエビデンスにすり替える攻撃

- アプリケーションのアテストーション前に、エビデンスに含まれるPCR値を改ざん・偽造



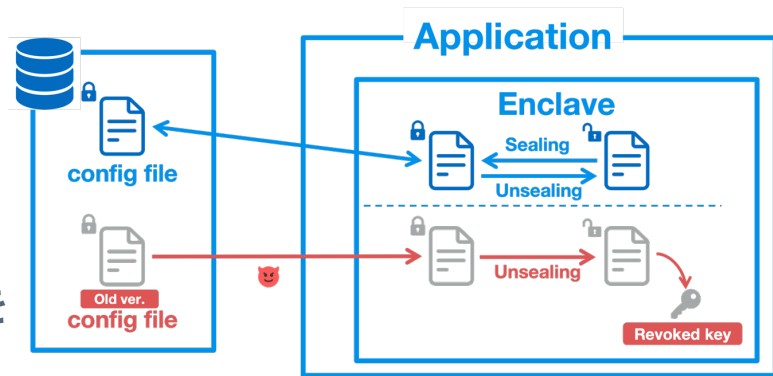
提案システムの攻撃モデル [2/3]

[3] Bryan Parno. Bootstrapping Trust in a "Trusted" Platform. In 3rd USENIX Workshop on Hot Topics in Security (HotSec 08), San Jose, CA, July 2008. USENIX Association.

Rollback Attack

攻撃者がシステムの正当な過去の状態を保存しておき、現在の最新状態をそれらで上書きして、不正な状態を正当だとシステムに誤認させる攻撃

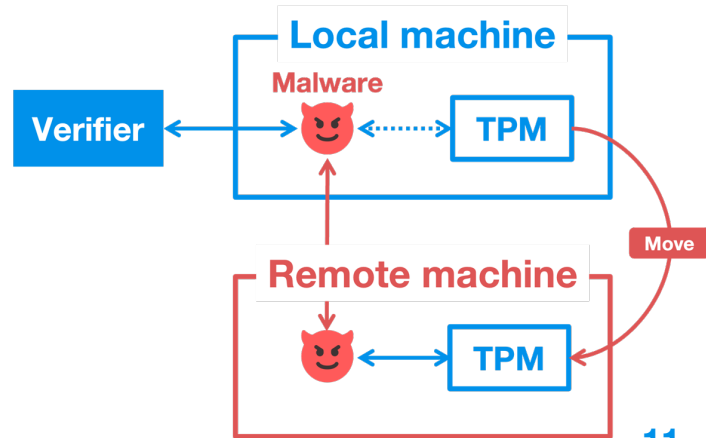
- 過去のconfig fileにあるエビデンスや失効鍵を正当なものとして使用するリスク



Cuckoo Attack^[3]

TPMの物理的挿し替え等により、攻撃者に制御される環境下にある正当なTPMへ中継されてしまう攻撃

- エビデンスの真正性をTPMの署名によって担保できないリスク



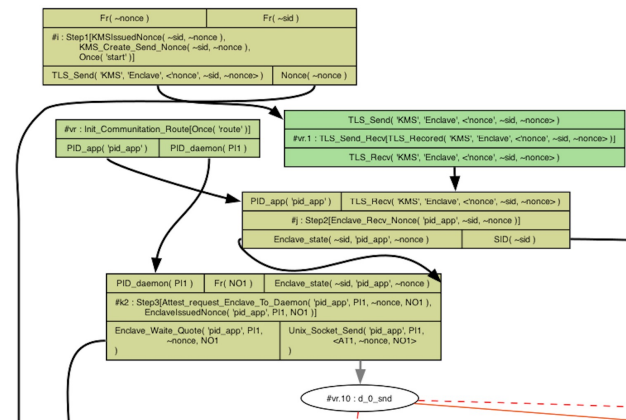
提案システムの攻撃モデル [3/3]

■ 各攻撃モデルに対応する安全性要件を設定

Security goal		Attack model
G1	Freshness of Evidence	Replay attack
G2	Integrity of Evidence	改ざん・偽造
G3	Freshness of Config file's state	Rollback attack
G4	Integrity of Config file's state	
G5	Integrity of Host code	Cuckoo attack
G6	Binding between Host Identity & Host code	
G7	Authenticity of Evidences	

Tamarinを用いた形式検証

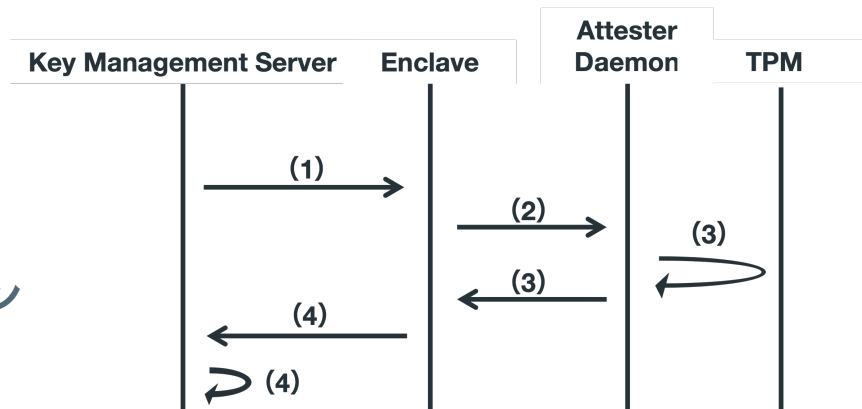
- Tamarin prover: セキュリティプロトコルを形式検証するためのツール
 - プロトコルの各エンティティの動作, Attack model, 検証対象のセキュリティ特性を, 項書き換え規則・一階述語論理式により記述
 - 満たすべき安全性の条件を論理式としてlemmaを記述
 - セキュリティ特性を破る攻撃トレースが反例として提示
 - 通信路の攻撃はDolev-Yaoモデル (改ざん・挿入・再送・破棄) ▼Traceの一部



提案プロトコルのモデル化

■ 検証対象プロトコル: ネットワーク接続時におけるアプリケーションのアテステーション

- (1) 鍵管理サーバからnonceを取得
- (2) エビデンス生成要求
- (3) エビデンス生成と返信
- (4) アプリケーションのアテステーション



■ プロトコルのモデル化

- 通信
 - Enclave-Daemon間のUnix domain socket通信は攻撃者に制御可能な通信
 - VerifierとEnclave間のTLS通信は状態遷移として簡素化
- アプリケーションのEnclave呼び出しは、Enclaveの状態遷移として抽象化
- EnclaveのRemote Attestationは先行研究^[5]より省略

質問

- exist-traceの書き方（Action factの引数について）
 - エビデンスにnonceを入れることでリプレイ攻撃を防ぐことを証明したい
 - Action factの引数に同一変数のnonceをいれてよいのか
 - 「同一のnonceを送受信している」モデル化になり，リプレイ攻撃を考慮したモデル化ができてないのでは
- ロールバック攻撃を想定しているため，プロトコル全体を何周かさせるモデル化をしたい
 - 状態爆発にならない方法について
 - 数回だけ周回させただけで，すべての場合について成り立つ，と言えるような帰納的なモデル化の仕方はあるのか